

DALI · DESIGN ENGINEERING 26F

Design Engineering

Session 2: Stack Choices & System Architecture

Petros Taxiarchis · October 1, 2026

Room B05

TODAY, AT A GLANCE

The whole run of show

0:00	Welcome back + roadmap
0:02	How a web app is actually built
0:07	Stack menu: frontend
0:12	Stack menu: backend
0:17	Stack menu: database, SQL vs. NoSQL
0:23	Stack menu: data access
0:27	Stack menu: state management
0:32	Stack menu: caching
0:37	Stack menu: type checking
0:41	Stack menu: backend-as-a-service & auth
0:45	Stack menu: hosting & deployment models
0:49	Stack menu: mobile, in one breath
0:51	How to choose
0:54	Choosing for agents specifically
0:57	Architecture Decision Records
0:59	Live demo: tear down a real app's stack
1:09	Lab: start your ADR + scaffold
1:19	Show & tell
1:21	Wrap-up

WELCOME BACK

0:00-0:02 · 2 min

LECTURE

Last time, this time

- Session 1: agentic coding & git, your safety net for letting an agent touch code
- Today: how a web app is actually put together, the full menu of stack choices, and you'll leave with a written decision for your own project

FOUNDATIONS

0:02–0:07 · 5 min

LECTURE

How a web app is actually built

- A URL is just an address. Typing one asks a question over the internet: "can I have this page?"
- **DNS** is the internet's phonebook. It turns a name like `dali.dartmouth.edu` into the actual address of a computer
- That request travels to a **server**: a computer that's always on, waiting to answer
- The server splits into two jobs: the **backend** decides what data to send back, the **database** is where that data actually lives when nothing is running
- Turning that response into the page you see is the **frontend**'s job, and it runs on your machine, not the server's
- Every click repeats this round trip. That's why a slow network makes an app feel sluggish no matter how good the code is

Frontend: what renders the page

- **React (Vite):** the most popular UI library. You build small, reusable pieces called components; Vite is just the tool that runs your code instantly while you develop, instead of making you wait
- **Next.js:** React, plus a built-in set of rules for routing and for rendering pages on the server before they reach the browser, decisions you'd otherwise have to make yourself
- **SvelteKit:** a lighter alternative to React. Less boilerplate to write, but a smaller community and fewer tutorials/examples for an agent to lean on
- **Plain HTML/JS:** no framework at all. Fine for something genuinely tiny, gets messy fast past a few interactive pages

THE STACK MENU

0:12–0:17 · 5 min

LECTURE

Backend: what decides what to send back

- **Node (Express):** JavaScript running on a server instead of just in the browser. Express is the classic, simple way to handle requests and send back responses
- **Next.js route handlers:** skip having a separate backend project at all, and write backend code right inside your frontend project
- **Python (FastAPI):** if your team already knows Python, a backend framework built to be fast and to auto-generate API documentation for you as you write it

THE STACK MENU

0:17–0:23 · 6 min

LECTURE

Database: SQL vs. NoSQL

A database is just organized, permanent storage. Your server's memory disappears when it restarts; a database doesn't.

SQL / relational (Postgres)

- Data lives in tables with fixed columns
- Rows connect to each other with keys (a student has many enrollments, a course has many students)
- Use it when your data has real relationships you need to query across, and you want the database itself to enforce the rules

NoSQL / document (MongoDB)

- Data lives in flexible, self-contained documents: one JSON-like blob per item
- No fixed schema required up front
- Use it when each record's shape varies a lot, or you don't know your schema yet

Default for this course: **Postgres**. Most student project data (users, posts, events, RSVPs) is relational, whether you planned it that way or not.

THE STACK MENU

0:23–0:27 · 4 min

LECTURE

Data access: how your code talks to the database

- **Raw SQL:** you write the exact query yourself. Full control, but easy to make mistakes, like forgetting to escape user input, which opens the door to SQL injection
- **Query builders:** write SQL-shaped code instead of raw query strings, a middle ground between the two
- **ORMs** (Drizzle, Prisma): treat database rows like regular code objects. Less SQL knowledge needed day to day
- Recommendation for this course: start with an ORM. The agent reads and writes it reliably, and its types catch mistakes before you run anything

THE STACK MENU

0:27–0:32 · 5 min

LECTURE

State management

"State" is whatever the screen currently remembers while the app runs: which tab is open, what's in a cart, whether a modal is showing.

- React's built-in `useState` handles most of what you need this course
- If several unrelated components need to share the same piece of state, lift it up to a shared parent instead of passing it down through five layers of props
- Data that comes from your API is a different problem: track its loading and error states on purpose, don't just treat it like regular state

THE STACK MENU

0:32-0:37 · 5 min

LECTURE

Caching

Caching means keeping a copy of something so you don't redo the work to get it again.

- **Browser cache:** images and scripts are stored locally so a repeat visit loads faster
- **Server-side cache:** store the result of a slow or expensive database query instead of recomputing it on every request
- Add it once a specific query is actually slow, or you're hitting the same data over and over. Don't add it before that: it trades speed for a new category of bugs, stale data

THE STACK MENU

0:37–0:41 · 4 min

LECTURE

Type checking

- **JavaScript:** no type checking at all. Anything goes while you're writing it, and mistakes show up later, at runtime, often in production
- **TypeScript:** adds type checking on top of JavaScript. Your editor and your agent both catch far more mistakes before the code ever runs
- Recommendation for this course: use TypeScript everywhere

THE STACK MENU

0:41-0:45 · 4 min

LECTURE

Backend-as-a-service & auth

- **Supabase** bundles backend, database, and auth together so you don't build those pieces from scratch. Faster to start, harder to move off of later
- Rolling your own login system means handling passwords and sessions correctly, yourself. Very easy to get wrong
- Use Supabase Auth instead. You're already on Supabase for the database, so it's one less thing to set up separately

THE STACK MENU

0:45–0:49 · 4 min

LECTURE

Hosting & deployment models

- **Vercel** and **Render** take your code and run it on the internet for you, so you're not managing a physical machine
- **Serverless** (Vercel): your backend only runs when a request actually comes in, and scales up and down automatically
- **Always-on server** (Render): simpler mental model, and necessary if you need background jobs or websockets
- Default for this course: Vercel, unless you specifically need an always-on server, then Render

THE STACK MENU

0:49–0:51 · 2 min

LECTURE

Mobile, in one breath

Expo / React Native lets you write once and ship a real iOS/Android app. It's only worth the setup cost if being an actual app-store app matters for your idea. Otherwise, a responsive web app that works fine on a phone's browser is enough, and far less to learn this term.

DECIDING

0:51–0:54 · 3 min

LECTURE

How to choose

- Constraints and team skills first
- The shape of your data (relational almost always wins for this course)
- Real, not imagined, scale. You are not Discord yet
- Cost, and lock-in
- Prototyping tools (Lovable, v0) are fast for a throwaway demo, but you don't own the architecture underneath

DECIDING

0:54–0:57 · 3 min

LECTURE

Choosing for agents specifically

- Popular, well-documented stacks get better agent output, since the model has seen far more of them
- TypeScript and typed schemas (Zod) give the agent error feedback it can actually act on, instead of guessing
- Conventions (file-based routing, migrations) reduce how much the agent has to guess about where things go

DECIDING

0:57–0:59 · 2 min

LECTURE

Architecture Decision Records

One page, written down, so future-you remembers why:

- **Context:** what are we deciding, and why does it matter
- **Options:** what we considered
- **Decision:** what we picked
- **Consequences:** what this costs us later

LIVE DEMO

0:59–1:09 · 10 min

LIVE DEMO

Tear down a real app's stack

1. Look up a popular app's publicly documented tech stack together (e.g. Discord's engineering blog)
2. Map each piece back to a category from today: frontend, backend, database, caching, etc.
3. Ask: why did they choose that, at their scale? (e.g. a custom database for billions of messages a day)
4. Ask: what would be overkill for your project right now? Boring, simple choices are correct until you have their problems

LAB

1:09–1:19 · 10 min

HANDS-ON

Lab: start your stack decision

1. Write `docs/adr-001-stack.md` for your project: at least 2 alternatives considered, and why you rejected them
2. Scaffold the chosen stack with the agent, in plan mode first

That's the whole lab. Diagram and deploy aren't in today's block.

SHOW & TELL

1:19–1:21 · 2 min

SHOW & TELL

Show & tell

- A couple of volunteers, not the whole room: there isn't time for everyone
- One sentence: your stack, and the one tradeoff that decided it

WRAP-UP

1:21–1:23 · 2 min

WRAP-UP

Next time

- Session 3: Context Engineering & Skills
- Bring your ADR and whatever scaffold you got to. No assigned homework, just pick up where you left off

Thank you!

Please stay for a moment, we're doing attendance in person.